

# Shuttle Dashboard — Technische Dokumentation

---

## Gruppe 5 · Java/JavaFX Projekt

---

### 1. Projektübersicht

Das Shuttle Dashboard ist eine JavaFX-Anwendung zur Steuerung und Überwachung des Übergabeprozesses einer Raumfähre. Es simuliert den vollständigen Ablauf nach der Landung: von der Sensor-Kalibrierung über die Reparatur defekter Komponenten bis zur Freigabe durch den Security Chief.

#### Kernfunktionen

| Funktion            | Beschreibung                                                     |
|---------------------|------------------------------------------------------------------|
| Phasenverwaltung    | Automatischer Ablauf: Landing → Sensor Loading → Operational     |
| Mission Control     | Reparaturaufgaben pro Shuttle-Teil, Technikerzuweisung, Freigabe |
| Logistics           | Bestellworkflow mit Genehmigung durch Security Chief             |
| Predictive Analysis | Trendauswertung über 5 Flughistorien; Frühwarnsystem             |
| Schedule            | Interaktiver 3-Tage-Übergabeplan mit Umplanungsfunktion          |
| Staff               | Mitarbeiterverwaltung und Teamübersicht                          |
| Inventory           | Lagerbestandsanzeige mit Statusfarbkodierung                     |
| History             | Protokoll abgeschlossener Wartungstickets                        |

#### Tech Stack

| Komponente    | Version |
|---------------|---------|
| Java          | 17      |
| JavaFX        | 21.0.2  |
| Build-Tool    | Maven   |
| UI-Definition | FXML    |

#### Einstiegspunkt

```
Main.java → App.java → main_view.fxml → MainController
```

---

### 2. Architektur

## Architekturmuster: MVC

Die Anwendung folgt dem **Model-View-Controller**-Muster mit strikter Schichtentrennung:

- **Model** (`model/`): Plain Java Objects (POJOs) — keine Logik, nur Daten
- **View** (`resources/view/*.fxml`): Deklarative UI-Definitionen in FXML
- **Controller** (`controller/`): Verbinden UI-Events mit Services; keine Datenzugriffslogik
- **Service** (`service/`): Geschäftslogik, Datenzugriff; alle als Singletons implementiert

## Package-Struktur

| Package                         | Klassen | Verantwortlichkeit                                    |
|---------------------------------|---------|-------------------------------------------------------|
| <code>com.group5.shuttle</code> | 2       | App-Einstieg ( <code>Main</code> , <code>App</code> ) |
| <code>controller</code>         | 11      | UI-Logik, Event-Handling                              |
| <code>service</code>            | 11      | Geschäftslogik, Datenzugriff                          |
| <code>model</code>              | 18      | Datenmodell (POJOs)                                   |
| <code>util</code>               | 2       | Querschnittsthemen (Farben, Tabellenzellen)           |

## Verwendete Entwurfsmuster

### Singleton

Alle Services halten genau eine Instanz über die gesamte Sitzung:

`TakeoverState`, `EmployeeService`, `InventoryService`, `SensorDataService`, `ScheduleService`, `FlightHistoryService`, `TicketStore`, `OrderStore`, `RoutineTaskStore`

```
// Beispiel: Singleton-Implementierung
public static InventoryService getInstance() {
    if (instance == null) instance = new InventoryService();
    return instance;
}
```

### Template Method

`BaseController` definiert die Navigationsmethode `loadView(fxml)`. Jede Subklasse implementiert `getNavigationButton()` — damit erhält `loadView` Zugriff auf das aktive Fenster:

```
// BaseController (abstrakte Vorlage)
protected void loadView(String fxml) {
    Parent root = new FXMLLoader(...).load();
    Stage stage = (Stage) getNavigationButton().getScene().getWindow();
    stage.setScene().setRoot(root);
}

// Subklasse (konkrete Implementierung)
@Override
protected Button getNavigationButton() { return btnBack; }
```

## Observer (JavaFX Properties)

`RepairTask` verwendet JavaFX Properties (`SimpleBooleanProperty`, `SimpleStringProperty`), damit Tabellenzeilen sich automatisch aktualisieren, wenn sich der Zustand ändert — ohne manuelles `refresh()`:

```
private final BooleanProperty done          = new
SimpleBooleanProperty(false);
private final StringProperty  partStatus  = new
SimpleStringProperty("NONE");
```

## 3. UML Klassendiagramm

```
classDiagram
    class BaseController {
        <>
        #loadView(fxml: String)
        #getNavigationButton()* Button
    }

    BaseController <|-- MainController
    BaseController <|-- MissionControlController
    BaseController <|-- TechnicianController
    BaseController <|-- InventoryController
    BaseController <|-- HistoryController
    BaseController <|-- StaffController
    BaseController <|-- LogisticsController
    BaseController <|-- ScheduleController

    MainController --> TakeoverState : nutzt
    MainController --> SensorDataService : nutzt
    MainController --> ScheduleService : nutzt
    MissionControlController --> TakeoverState : nutzt
    MissionControlController --> InventoryService : nutzt
    MissionControlController --> ScheduleService : nutzt
    LogisticsController --> InventoryService : nutzt
    LogisticsController --> OrderStore : nutzt
    TechnicianController --> SensorDataService : nutzt

    class TakeoverState {
        <>
        -instance: TakeoverState
        +getInstance()$ TakeoverState
        +getAppPhase() AppPhase
        +setOperational()
        +approve(partKey, chiefName)
        +markTechnicianDone(partKey, techName)
        +generateRepairs(data, thresholds, sds)
```

```

        +getProgress() double
        +isTakeoverComplete() boolean
        +reset()
    }

    class SensorDataService {
        <>
        +getInstance()$ SensorDataService
        +loadSensorData() ShuttleData
        +loadThresholds() Map
        +evaluate(value, threshold) String
        +loadRoutineTasks() List~RoutineTask~
    }

    class InventoryService {
        <>
        +getInstance()$ InventoryService
        +loadInventory() List~InventoryItem~
        +saveInventory(items)
    }

    class EmployeeService {
        <>
        +getInstance()$ EmployeeService
        +getAllEmployees() List~Employee~
        +getId(id) Employee
        +getByRole(role) List~Employee~
    }

    class ScheduleService {
        <>
        +getInstance()$ ScheduleService
        +loadSchedule() TakeoverSchedule
    }

    class PredictiveAnalysisService {
        +analyzeAll() List~TrendResult~
    }

    TakeoverState --> RepairTask : verwaltet
    TakeoverState --> EmployeeService : nutzt
    MissionControlController --> EmployeeService : nutzt
    MainController --> PredictiveAnalysisService : nutzt
    PredictiveAnalysisService --> FlightHistoryService : nutzt
    PredictiveAnalysisService --> SensorDataService : nutzt

    class StatusColors {
        <>
        +forSensorStatus(status)$ String
        +forStockStatus(status)$ String
        +forOrderStatus(status)$ String
    }

    class ColoredTableCell~T~ {

```

```

        -colorMapper: Function~String,String~
        +ColoredTableCell(colorMapper)
        +updateItem(item, empty)
    }

    class RepairTask {
        -sensorName: String
        -status: String
        -action: String
        -done: BooleanProperty
        -partStatus: StringProperty
        +doneProperty() BooleanProperty
        +setDone(done)
    }

    class InventoryItem {
        -name: String
        -quantity: int
        -status: String
        -part: String
    }

    class Employee {
        -id: String
        -name: String
        -role: String
        -team: String
    }

    class LogisticsOrder {
        -orderNumber: String
        -partName: String
        -quantity: int
        -status: String
        +approve(approver)
        +markOrdered()
        +markDelivered()
        +reject(approver)
    }

```

## 4. UML Sequenzdiagramm — Takeover-Workflow

Zeigt den vollständigen Ablauf vom App-Start bis zur Freigabe eines Shuttle-Teils durch den Security Chief.

```

sequenceDiagram
    participant OS as Betriebssystem
    participant App
    participant Main as MainController
    participant TS as TakeoverState
    participant SDS as SensorDataService

```

```
participant MCC as MissionControlController
participant IS as InventoryService

OS->>App: main()
App->>App: launch(App.class)
App->>Main: initialize() [FXML geladen]
Main->>TS: getInstance()
TS-->>Main: LANDING Phase aktiv
Main->>Main: startLandingTimer(15s)

Note over Main,TS: Nach 15 Sekunden automatisch...
Main->>TS: beginSensorLoading()
Main->>Main: startSensorLoadingTimer(10s)

Note over Main,TS: Nach 10 Sekunden automatisch...
Main->>SDS: loadSensorData()
SDS-->>Main: ShuttleData
Main->>SDS: loadThresholds()
SDS-->>Main: Map(part → sensor → threshold)
Main->>TS: setOperational()
Main->>TS: generateRepairs(data, thresholds, sds)
TS-->>Main: RepairTasks für Orbiter, SRB, External Tank

Main-->>User: Dashboard zeigt Sensor-Warnungen

User->>Main: Klick "Mission Control"
Main->>MCC: loadView("mission_control.fxml")
MCC->>TS: getRepairs("orbiter")
MCC->>IS: loadInventory()
MCC-->>User: Zeige Reparaturtabelle + Inventar

User->>MCC: selectPart("orbiter")
User->>MCC: Wähle Techniker aus ComboBox
User->>MCC: Klick "Confirm"
MCC->>TS: registerWorker("orbiter", name, "Technician")
MCC-->>User: Zeige Arbeits-Panel

loop Für jede Reparaturaufgabe
    User->>MCC: Klick "Done" Checkbox
    MCC->>IS: loadInventory()
    MCC->>IS: saveInventory() [qty - 1]
    MCC->>TS: RepairTask.setDone(true)
end

User->>MCC: Klick "Technician Done"
MCC->>TS: markTechnicianDone("orbiter", techName)
MCC->>TS: logRepairsToHistory()
TS-->>MCC: technicianDone["orbiter"] = true

User->>MCC: Wähle Security Chief aus ComboBox
User->>MCC: Klick "Confirm"
MCC->>TS: registerWorker("orbiter", name, "Security Chief")
MCC->>TS: canApprove("orbiter")
TS-->>MCC: true [Techniker fertig, noch nicht freigegeben]
```

```
User->>MCC: Klick "Give Security Chief OK"
MCC->>TS: approve("orbiter", chiefName)
TS-->>MCC: securityApprovals["orbiter"] = true
MCC-->>User: Header: "Orbiter – APPROVED ✓" (grün)

User->>MCC: Klick "Back"
MCC->>Main: loadView("main_view.fxml")
Main->>TS: getProgress()
TS-->>Main: 0.33 (1 von 3 Teilen)
Main->>TS: isTakeoverComplete()
TS-->>Main: false

Note over Main,TS: Ablauf für SRB und External Tank wiederholen...
Note over Main,TS: Nach 3. Freigabe: isTakeoverComplete() = true
Main-->>User: "Takeover Complete" Button erscheint
```

## 5. Programmlogik — Kernprozesse

### 5.1 Phasenverwaltung (TakeoverState)

Die Anwendung durchläuft drei Phasen, die automatisch durch JavaFX-Timer (PauseTransition) gesteuert werden:

| Phase          | Dauer                                 | Zugriff       |
|----------------|---------------------------------------|---------------|
| LANDING        | 15 Sekunden (= 15 simulierte Minuten) | Nur Logistics |
| SENSOR_LOADING | 10 Sekunden (= 30 simulierte Minuten) | Eingeschränkt |
| OPERATIONAL    | Unbegrenzt                            | Vollzugriff   |

### 5.2 Reparatur-Workflow (pro Shuttle-Teil)

```
1. Techniker wählt Teil → bestätigt Identität
2. System zeigt Reparaturaufgaben (Sensor-Status: WARNING / REPLACE)
3. Pro Aufgabe: Lagerbestand prüfen
   → IN_STOCK: Checkbox abhaken → Bestand -1
   → OUT_OF_STOCK: Bestellung aufgeben → 10s Liefersimulation → ARRIVED
4. Alle Aufgaben abgehakt → Klick "Technician Done"
   → TakeoverState schreibt Maintenance-Tickets
5. Security Chief logt sich ein → prüft Reparaturen → Klick "Approve"
6. securityApprovals["part"] = true → Progress +33%
7. Nach 3 Teilen: isTakeoverComplete() = true → Takeover abgeschlossen
```

### 5.3 Logistics-Bestellworkflow (LogisticsOrder)

Bestellungen durchlaufen einen mehrstufigen Genehmigungsprozess:

```
PENDING_APPROVAL → (Approve) → ORDERED → (10s) → DELIVERED
                  → (Reject) → REJECTED
```

- Jeder Mitarbeiter kann eine Bestellung aufgeben
- Security Chief muss genehmigen
- Nach Genehmigung: 10-Sekunden-Liefersimulation
- Bei Lieferung: Lagerbestand wird automatisch erhöht

## 5.4 Prädiktive Analyse (`PredictiveAnalysisService`)

Der Service wertet 5 gespeicherte Flughistorien aus:

1. Für jeden Sensor: Werte der letzten 5 Flüge auslesen
2. Trend berechnen: letzter Wert vs. Durchschnitt der ersten 4
3. Klassifikation: `RISING` / `FALLING` / `STABLE`
4. Bei kritischem Trend + aktuell `WARNING/REPLACE`: Frühwarnung im Dashboard

## 5.5 Schedule-Verwaltung (`ScheduleController`)

- 3-Tage-Plan wird aus `schedule.json` geladen
- Jede Zeile ist editierbar: Mitarbeiter kann per ComboBox umgeplant werden
- Änderungen sind nur für die aktuelle Sitzung gültig (transient)

---

# 6. Setup & Ausführung

## Voraussetzungen

- Java 17 oder höher
- Maven 3.6+

## Starten

```
cd shuttle-dashboard
mvn clean javafx:run
```

## Kompilieren ohne Ausführen

```
mvn compile
```

## Projektstruktur

```
shuttle-dashboard/
├─ pom.xml
```



```

└─ src/main/
    └─ java/com/group5/shuttle/
        └─ App.java          ← JavaFX-Einstiegspunkt
        └─ Main.java         ← OS-Einstiegspunkt (Wrapper)
        └─ controller/      ← 11 Controller (extends
BaseController)
            └─ service/      ← 11 Services (Singletons)
            └─ model/        ← 18 Datenklassen (POJOs)
            └─ util/        ← StatusColors,
ColoredTableCell
    └─ resources/view/
        └─ main_view.fxml    ← Haupt-Dashboard
        └─ mission_control.fxml ← Reparatur & Freigabe
        └─ logistics.fxml    ← Bestellworkflow
        └─ schedule_view.fxml ← 3-Tage-Plan
        └─ technician.fxml   ← Sensor-Übersicht
        └─ staff.fxml        ← Mitarbeiterverwaltung
        └─ inventory.fxml    ← Lagerbestand
        └─ history.fxml      ← Wartungsprotokoll

```